Assignment 10:

```
/**
 *
================================================================================
 * KBTCOE Connect — Opportunity Explorer Application Logic (app.js)
 * Assignment 10: Fetch API & JSON
 *
 * Demonstrates:
 * 1. Asynchronous JavaScript with async / await and Promises
 * 2. Fetch API: requesting local JSON resources
 * 3. HTTP status inspection via response.ok and response.status
 * 4. JSON parsing with response.json() and defensive data validation
 * 5. Complete state machine: Loading, Success, Empty Data, Empty Filter, and
Error states
 * 6. Error handling with try...catch and interactive Retry capability
 * 7. Multi-criteria client-side search and category filtering
 * 8. Safe DOM manipulation via createElement and textContent (preventing XSS)
 * 9. CORS / file:// protocol detection with actionable developer guidance
 *
================================================================================
 */

"use strict";

//
================================================================================
// 1. APPLICATION STATE
//
================================================================================
const state = {
  opportunities: [],          // Raw dataset parsed from JSON
  filteredOpportunities: [],  // Filtered dataset currently displayed
  searchQuery: "",            // Active search term
  selectedType: "all",        // Active opportunity type filter
  fetchStatus: "idle",        // 'idle' | 'loading' | 'success' | 'empty' |
'error'
  errorMessage: "",           // Last recorded error description
  dataSourceUrl: "./data/opportunities.json" // Configurable data endpoint
};

//
================================================================================
// 2. DATA VALIDATION (Defensive programming)
//
================================================================================
/**
 * Validates that the parsed JSON conforms to expected opportunity array schema.
 * @param {any} data - Parsed JSON object
 * @returns {{ isValid: boolean, message: string }}
 */
function validateOpportunitySchema(data) {
  if (!Array.isArray(data)) {
    return {
```

```
      isValid: false,
      message: "Malformed JSON payload: Root element must be an array of
opportunities."
    };
  }

  for (let i = 0; i < data.length; i++) {
    const item = data[i];
    if (!item.id || !item.title || !item.type || !item.organization) {
      return {
        isValid: false,
        message: `Opportunity record at index ${i} is missing mandatory fields
(id, title, type, organization).`
      };
    }
  }

  return { isValid: true, message: "" };
}

//
================================================================================
// 3. ASYNCHRONOUS FETCH ENGINE
//
================================================================================
/**
 * Fetches JSON data asynchronously using modern Fetch API and async/await.
 * Enforces checking response.ok and handling network/parsing exceptions.
 *
 * @param {string} url - Target JSON endpoint URL
 */
async function loadOpportunities(url = state.dataSourceUrl) {
  state.fetchStatus = "loading";
  state.errorMessage = "";
  renderCurrentState();

  const refreshBtn = document.getElementById("btnRefresh");
  if (refreshBtn) {
    refreshBtn.disabled = true;
    refreshBtn.setAttribute("aria-busy", "true");
  }

  try {
    console.log(`[Fetch API] Initiating HTTP GET request to: ${url}`);

    // Simulate realistic 300ms network latency for smooth UI transitions
    await new Promise(resolve => setTimeout(resolve, 300));

    // 1. Perform HTTP request via Fetch API (returns a Promise)
    const response = await fetch(url, {
      headers: {
        "Accept": "application/json"
      }
    });
```

```
    console.log(`[Fetch API] Response status: ${response.status}
(${response.statusText || "OK"})`);

    // 2. Check HTTP Response status via response.ok
    // NOTE: A fulfilled fetch() promise only means network contact succeeded.
    // HTTP errors like 404 Not Found or 500 Server Error still resolve the
promise!
    // We MUST check response.ok (status 200–299) to verify logical success.
    if (!response.ok) {
      throw new Error(`HTTP Error ${response.status}: ${response.statusText ||
"Failed to load resource."}`);
    }

    // 3. Parse JSON stream into JavaScript objects (returns a Promise)
    const data = await response.json();

    // 4. Validate data shape defensively
    const validation = validateOpportunitySchema(data);
    if (!validation.isValid) {
      throw new Error(validation.message);
    }

    // 5. Update state
    state.opportunities = data;

    if (data.length === 0) {
      state.fetchStatus = "empty";
    } else {
      state.fetchStatus = "success";
    }

    console.log(`[Fetch API] Successfully loaded and validated ${data.length}
opportunity records.`);

  } catch (error) {
    // 6. Graceful error handling across network failures, 404s, and parse
errors
    console.error("[Fetch API] Request encountered an error:", error);
    state.fetchStatus = "error";
    state.errorMessage = error.message || "An unexpected network error occurred
while retrieving opportunities.";
  } finally {
    if (refreshBtn) {
      refreshBtn.disabled = false;
      refreshBtn.removeAttribute("aria-busy");
    }
    // Apply filters and render results
    applyFilters();
  }
}

//
================================================================================
```

```
// 4. MULTI-CRITERIA FILTERING & SEARCH
//
================================================================================
/**
 * Pure helper function to filter opportunities by search query and type.
 *
 * @param {Array<Object>} list - Complete opportunity list
 * @param {string} query - Keyword search term
 * @param {string} typeFilter - Selected opportunity type ('all' or specific
type)
 * @returns {Array<Object>} Filtered array
 */
function filterOpportunitiesLogic(list, query = "", typeFilter = "all") {
  if (!Array.isArray(list)) return [];

  const normalizedQuery = query.trim().toLowerCase();
  const normalizedType = typeFilter.toLowerCase();

  return list.filter(item => {
    // 1. Type matching
    const matchesType = (normalizedType === "all" || item.type.toLowerCase() ===
normalizedType);

    // 2. Query matching (title, organization, description, or tags)
    if (!normalizedQuery) {
      return matchesType;
    }

    const matchesTitle = item.title.toLowerCase().includes(normalizedQuery);
    const matchesOrg =
item.organization.toLowerCase().includes(normalizedQuery);
    const matchesDesc = (item.description ||
"").toLowerCase().includes(normalizedQuery);
    const matchesTags = Array.isArray(item.tags) && item.tags.some(t =>
t.toLowerCase().includes(normalizedQuery));

    const matchesSearch = matchesTitle || matchesOrg || matchesDesc ||
matchesTags;

    // Concurrently satisfy both type and search filters
    return matchesType && matchesSearch;
  });
}

/**
 * Applies active search and type filters to state and updates UI.
 */
function applyFilters() {
  if (state.fetchStatus === "loading" || state.fetchStatus === "error") {
    renderCurrentState();
    return;
  }

  state.filteredOpportunities = filterOpportunitiesLogic(
```

```
    state.opportunities,
    state.searchQuery,
    state.selectedType
  );

  renderCurrentState();
}

//
================================================================================
// 5. SAFE DOM RENDERING & UI STATE CONTROLLERS
//
================================================================================

/**
 * Creates an Opportunity Card element using safe DOM methods (createElement &
textContent).
 * Strictly avoids untrusted innerHTML injection.
 *
 * @param {Object} opp - Opportunity record
 * @returns {HTMLElement} Card article element
 */
function createOpportunityCardElement(opp) {
  const card = document.createElement("article");
  card.className = "opportunity-card";
  card.setAttribute("data-opp-id", opp.id);

  // --- Top Row: Type Badge & Location ---
  const topRow = document.createElement("div");
  topRow.className = "card-top-row";

  const typeBadge = document.createElement("span");
  const typeLower = (opp.type || "default").toLowerCase();
  typeBadge.className = `type-badge badge-${typeLower}`;
  typeBadge.textContent = opp.type;

  const locationEl = document.createElement("span");
  locationEl.className = "opp-location";
  locationEl.textContent = `📍 ${opp.location || "Campus / Remote"}`;

  topRow.appendChild(typeBadge);
  topRow.appendChild(locationEl);

  // --- Header: Title & Organization ---
  const titleEl = document.createElement("h3");
  titleEl.className = "opp-title";
  titleEl.textContent = opp.title;

  const orgEl = document.createElement("p");
  orgEl.className = "opp-organization";
  orgEl.textContent = `🏛 ${opp.organization}`;

  // --- Description ---
  const descEl = document.createElement("p");
```

```
descEl.className = "opp-description";
descEl.textContent = opp.description;

// --- Key Details Box ---
const detailsBox = document.createElement("div");
detailsBox.className = "opp-details-box";

// Eligibility
const eligRow = document.createElement("div");
eligRow.className = "detail-row";
const eligLabel = document.createElement("span");
eligLabel.className = "detail-label";
eligLabel.textContent = "Eligibility:";
const eligVal = document.createElement("span");
eligVal.className = "detail-value";
eligVal.textContent = opp.eligibility;
eligRow.appendChild(eligLabel);
eligRow.appendChild(eligVal);

// Reward / Stipend
const rewardRow = document.createElement("div");
rewardRow.className = "detail-row";
const rewardLabel = document.createElement("span");
rewardLabel.className = "detail-label";
rewardLabel.textContent = "Reward / Perks:";
const rewardVal = document.createElement("span");
rewardVal.className = "detail-value reward-highlight";
rewardVal.textContent = opp.reward || "Certificate of Excellence";
rewardRow.appendChild(rewardLabel);
rewardRow.appendChild(rewardVal);

// Deadline
const deadlineRow = document.createElement("div");
deadlineRow.className = "detail-row";
const deadlineLabel = document.createElement("span");
deadlineLabel.className = "detail-label";
deadlineLabel.textContent = "Sample Deadline:";
const deadlineVal = document.createElement("span");
deadlineVal.className = "detail-value deadline-highlight";
deadlineVal.textContent = opp.deadline;
deadlineRow.appendChild(deadlineLabel);
deadlineRow.appendChild(deadlineVal);

detailsBox.appendChild(eligRow);
detailsBox.appendChild(rewardRow);
detailsBox.appendChild(deadlineRow);

// --- Tags Section ---
const tagsWrap = document.createElement("div");
tagsWrap.className = "opp-tags-wrapper";

if (Array.isArray(opp.tags)) {
  opp.tags.forEach(tag => {
    const tagPill = document.createElement("span");
```

```
      tagPill.className = "tag-pill";
      tagPill.textContent = tag;
      tagsWrap.appendChild(tagPill);
    });
  }

  // --- Footer Action Row ---
  const cardFooter = document.createElement("div");
  cardFooter.className = "card-footer-row";

  const btnApply = document.createElement("button");
  btnApply.type = "button";
  btnApply.className = "btn-apply-demo";
  btnApply.textContent = "View & Apply (Demonstration) →";
  btnApply.addEventListener("click", () => {
    alert(`[Demonstration Notice]\nOpportunity: "${opp.title}"\nOrganizer:
${opp.organization}\nDeadline: ${opp.deadline}\n\nThis is an educational
frontend demonstration. Real application pipelines are integrated in later
capstone practicals.`);
  });

  cardFooter.appendChild(btnApply);

  // Assemble Card
  card.appendChild(topRow);
  card.appendChild(titleEl);
  card.appendChild(orgEl);
  card.appendChild(descEl);
  card.appendChild(detailsBox);
  card.appendChild(tagsWrap);
  card.appendChild(cardFooter);

  return card;
}

/**
 * Primary UI State Machine renderer.
 */
function renderCurrentState() {
  const container = document.getElementById("opportunitiesContainer");
  const countEl = document.getElementById("resultsCount");
  const statusEl = document.getElementById("filterStatus");

  if (!container) return;
  container.replaceChildren();

  // --- 1. LOADING STATE ---
  if (state.fetchStatus === "loading") {
    const loadingBox = document.createElement("div");
    loadingBox.className = "state-card state-loading";
    loadingBox.setAttribute("role", "status");
    loadingBox.setAttribute("aria-live", "polite");

    const spinner = document.createElement("div");
```

```
    spinner.className = "loading-spinner";

    const text = document.createElement("p");
    text.className = "state-title";
    text.textContent = "Fetching opportunities via Fetch API...";

    const subtext = document.createElement("p");
    subtext.className = "state-desc";
    subtext.textContent = "Resolving Promise stream and validating JSON
schema...";

    loadingBox.appendChild(spinner);
    loadingBox.appendChild(text);
    loadingBox.appendChild(subtext);
    container.appendChild(loadingBox);

    if (countEl) countEl.textContent = "Loading...";
    if (statusEl) statusEl.textContent = "Status: Waiting for HTTP response...";
    return;
  }

  // --- 2. ERROR STATE (with functional Retry button) ---
  if (state.fetchStatus === "error") {
    const errorBox = document.createElement("div");
    errorBox.className = "state-card state-error";
    errorBox.setAttribute("role", "alert");
    errorBox.setAttribute("aria-live", "assertive");

    const icon = document.createElement("span");
    icon.className = "state-icon";
    icon.textContent = "⚠";

    const title = document.createElement("h3");
    title.className = "state-title error-title";
    title.textContent = "Failed to Fetch Opportunities";

    const desc = document.createElement("p");
    desc.className = "state-desc";
    desc.textContent = state.errorMessage;

    const retryBtn = document.createElement("button");
    retryBtn.type = "button";
    retryBtn.className = "btn-retry";
    retryBtn.textContent = "🔄 Retry Fetching";
    retryBtn.addEventListener("click", () => {
      // Re-trigger fetch on click
      loadOpportunities(state.dataSourceUrl);
    });

    errorBox.appendChild(icon);
    errorBox.appendChild(title);
    errorBox.appendChild(desc);
    errorBox.appendChild(retryBtn);
    container.appendChild(errorBox);
```

```
    if (countEl) countEl.textContent = "0";
    if (statusEl) statusEl.textContent = "Status: Request failed";
    return;
  }

  // --- 3. EMPTY RAW DATA STATE ---
  if (state.fetchStatus === "empty" || state.opportunities.length === 0) {
    const emptyBox = document.createElement("div");
    emptyBox.className = "state-card state-empty";

    const icon = document.createElement("span");
    icon.className = "state-icon";
    icon.textContent = "📭";

    const title = document.createElement("h3");
    title.className = "state-title";
    title.textContent = "No Opportunities Found in JSON Source";

    const desc = document.createElement("p");
    desc.className = "state-desc";
    desc.textContent = "The opportunities.json file loaded successfully but
currently contains 0 records.";

    emptyBox.appendChild(icon);
    emptyBox.appendChild(title);
    emptyBox.appendChild(desc);
    container.appendChild(emptyBox);

    if (countEl) countEl.textContent = "0";
    if (statusEl) statusEl.textContent = "Status: Data file is empty";
    return;
  }

  // --- 4. EMPTY FILTER RESULTS STATE ---
  if (state.filteredOpportunities.length === 0) {
    const noMatchBox = document.createElement("div");
    noMatchBox.className = "state-card state-empty";

    const icon = document.createElement("span");
    icon.className = "state-icon";
    icon.textContent = "🔍";

    const title = document.createElement("h3");
    title.className = "state-title";
    title.textContent = "No Matching Opportunities Found";

    const desc = document.createElement("p");
    desc.className = "state-desc";
    desc.textContent = state.searchQuery
      ? `No records match query "${state.searchQuery}" in category
"${state.selectedType}". Try adjusting your keywords or clearing the filters.`
      : `No opportunities available in category "${state.selectedType}".`;
```

```
    const resetBtn = document.createElement("button");
    resetBtn.type = "button";
    resetBtn.className = "btn-reset-filters";
    resetBtn.textContent = "Reset All Filters";
    resetBtn.addEventListener("click", handleResetFilters);

    noMatchBox.appendChild(icon);
    noMatchBox.appendChild(title);
    noMatchBox.appendChild(desc);
    noMatchBox.appendChild(resetBtn);
    container.appendChild(noMatchBox);

    if (countEl) countEl.textContent = "0";
    if (statusEl) statusEl.textContent = `Showing 0 of
${state.opportunities.length} opportunities`;
    return;
  }

  // --- 5. SUCCESS STATE (Render Grid of Cards) ---
  const fragment = document.createDocumentFragment();
  state.filteredOpportunities.forEach(opp => {
    const cardEl = createOpportunityCardElement(opp);
    fragment.appendChild(cardEl);
  });
  container.appendChild(fragment);

  // Update counts
  if (countEl) countEl.textContent = String(state.filteredOpportunities.length);
  if (statusEl) {
    statusEl.textContent = `Showing ${state.filteredOpportunities.length} of
${state.opportunities.length} opportunities`;
  }
}

//
================================================================================
// 6. EVENT HANDLERS
//
================================================================================
function handleSearchInput(e) {
  state.searchQuery = e.target.value;
  applyFilters();
}

function handleTypeChange(e) {
  state.selectedType = e.target.value;
  applyFilters();
}

function handleResetFilters() {
  state.searchQuery = "";
  state.selectedType = "all";

  const searchInput = document.getElementById("searchInput");
```

```
  const typeSelect = document.getElementById("typeFilter");

  if (searchInput) searchInput.value = "";
  if (typeSelect) typeSelect.value = "all";

  applyFilters();
}

function handleSimulateError() {
  // Simulates an HTTP 404 error to demonstrate error handling & retry
capability
  console.warn("[Simulation] Intentionally fetching non-existent endpoint to
verify error handling.");
  loadOpportunities("./data/non-existent-file-404.json");
}

//
================================================================================
// 7. INITIALIZATION & CORS ENVIRONMENT CHECK
//
================================================================================
function initOpportunityExplorer() {
  console.log("Initializing KBTCOE Connect Opportunity Explorer (Assignment
10)...");

  // Protocol Check: Alert student if launched via file://
  if (typeof window !== "undefined" && window.location.protocol === "file:") {
    const noticeEl = document.getElementById("corsProtocolNotice");
    if (noticeEl) {
      noticeEl.removeAttribute("hidden");
    }
  }

  // Attach Event Listeners
  const searchInput = document.getElementById("searchInput");
  if (searchInput) {
    searchInput.addEventListener("input", handleSearchInput);
  }

  const typeFilter = document.getElementById("typeFilter");
  if (typeFilter) {
    typeFilter.addEventListener("change", handleTypeChange);
  }

  const btnReset = document.getElementById("btnReset");
  if (btnReset) {
    btnReset.addEventListener("click", handleResetFilters);
  }

  const btnRefresh = document.getElementById("btnRefresh");
  if (btnRefresh) {
    btnRefresh.addEventListener("click", () => {
      state.dataSourceUrl = "./data/opportunities.json";
      loadOpportunities();
```

```
    });
  }

  const btnSimError = document.getElementById("btnSimulateError");
  if (btnSimError) {
    btnSimError.addEventListener("click", handleSimulateError);
  }

  // Initial Data Fetch
  loadOpportunities();
}

// Run when DOM is ready
if (typeof document !== "undefined") {
  if (document.readyState === "loading") {
    document.addEventListener("DOMContentLoaded", initOpportunityExplorer);
  } else {
    initOpportunityExplorer();
  }
}

// Export for Node.js automated unit testing
if (typeof module !== "undefined" && module.exports) {
  module.exports = {
    validateOpportunitySchema,
    filterOpportunitiesLogic,
    state
  };
}
```